

머신러닝 및 실습

(Lecture 11)

노연수

(esnoh@koreatech.ac.kr)

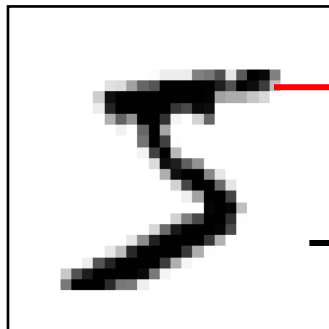
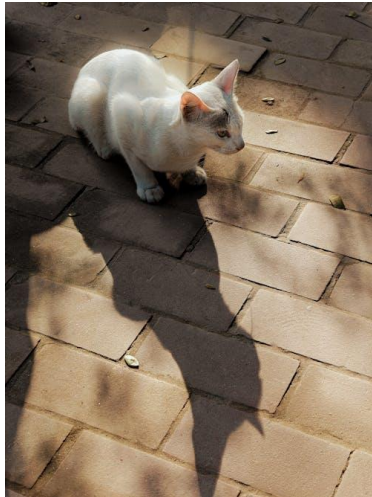
Course Plan

Week	Lecture	Contents	Notes
1	-	-	대체휴일
2	파이썬 기초 I	자료형, 연산자, 조건문, 반복문	
3	파이썬 기초 II	함수, 모듈, 패키지, 클래스	
4	머신러닝이란 무엇인가?	머신러닝 기초 개념, 머신러닝 시스템의 종류 등	
5	머신러닝의 전체적인 흐름 이해하기 I	프로젝트 방향 수립, 데이터 수집, 데이터 분석	
6	머신러닝의 전체적인 흐름 이해하기 II	데이터 전처리, 모델 학습, 모델 튜닝	
7	분류모델의 이해	이진 분류기 훈련, 성능 측정, 다중 분류	
8	중간고사	-	
9	회귀모델의 이해	선형 회귀, 경사 하강법	프로젝트 공지
10	회귀모델의 확장	다항 회귀, 로지스틱 회귀	
11	서포트 벡터 머신의 이해	규제 선형 모델, SVM 분류/회귀 등	
12	다양한 분류 모델과 앙상블	결정 트리, 랜덤 포레스트	
13	차원 축소	차원 축소, 주성분 분석(PCA)	이러닝(5/25)
14	군집 분석	K-Means clustering	
15	프로젝트 발표	프로젝트 발표	
16	보강, 기말고사	보강(Week 1), 기말고사(미정)	

차원 축소(Dimensionality Reduction)

데이터 압축(1)

- ▶ 주어진 데이터의 정보 손실을 최소화 하면서 데이터를 축소



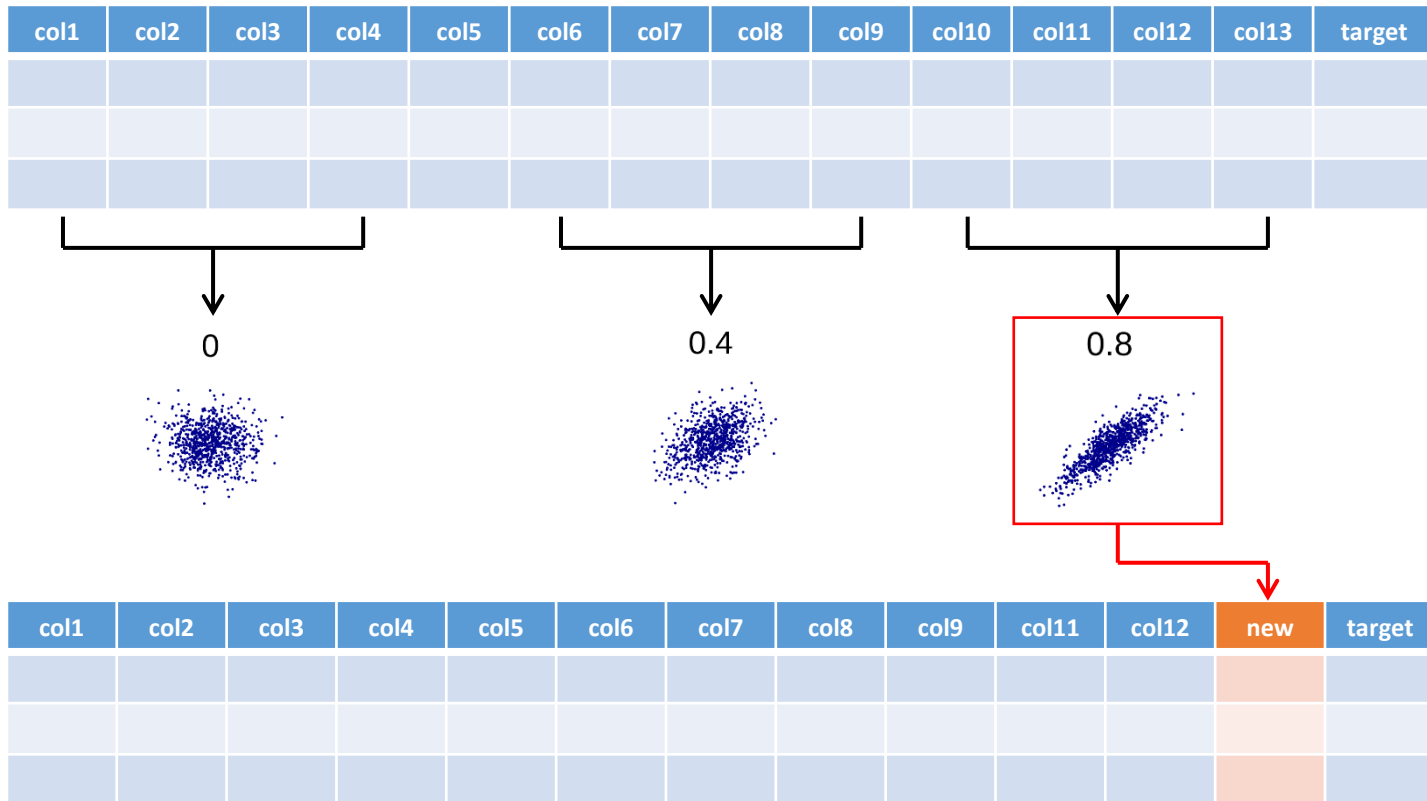
중요한 특징 → 특징 추출(feature extraction)!

상대적으로 중요하지 않은 특징

차원 축소(Dimensionality Reduction)

데이터 압축(2)

- 상관관계(correlation)가 높은 특징들은 하나의 특징으로 변환할 수 있음

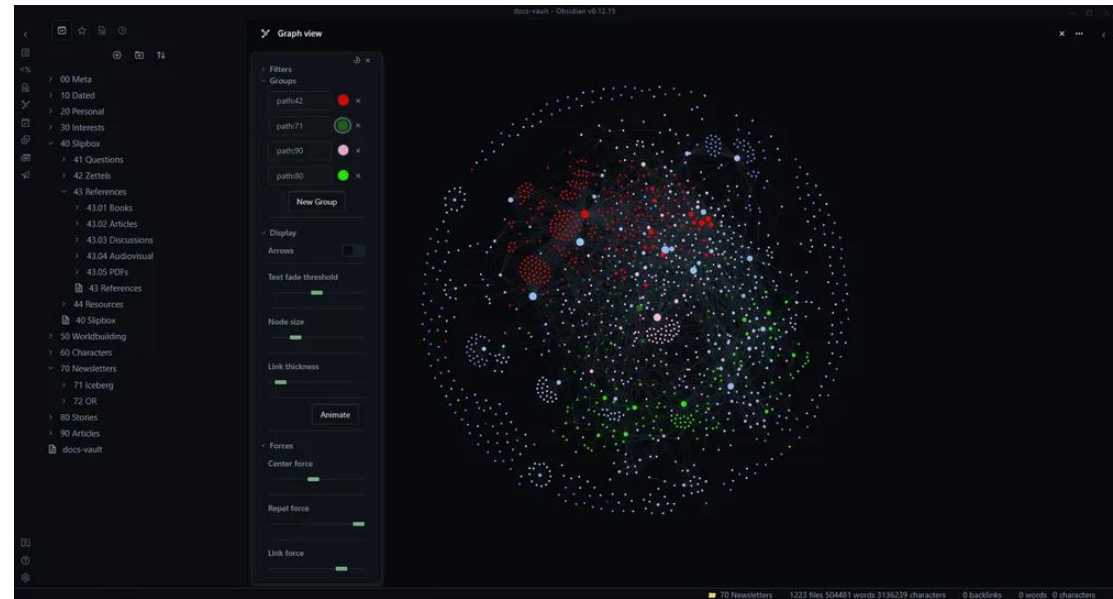
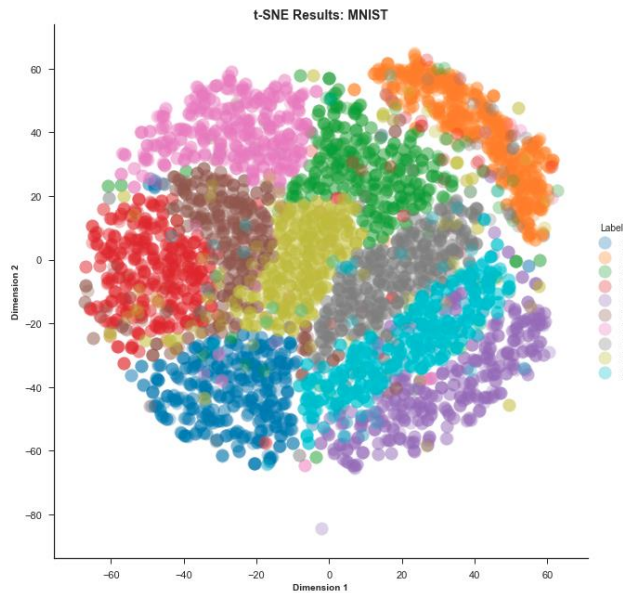


중요한 특징은 유지하면서 훈련 속도 향상

차원 축소(Dimensionality Reduction)

데이터 시각화

▶ 고차원 데이터를 저차원의 압축된 그래프로 그릴 수 있음

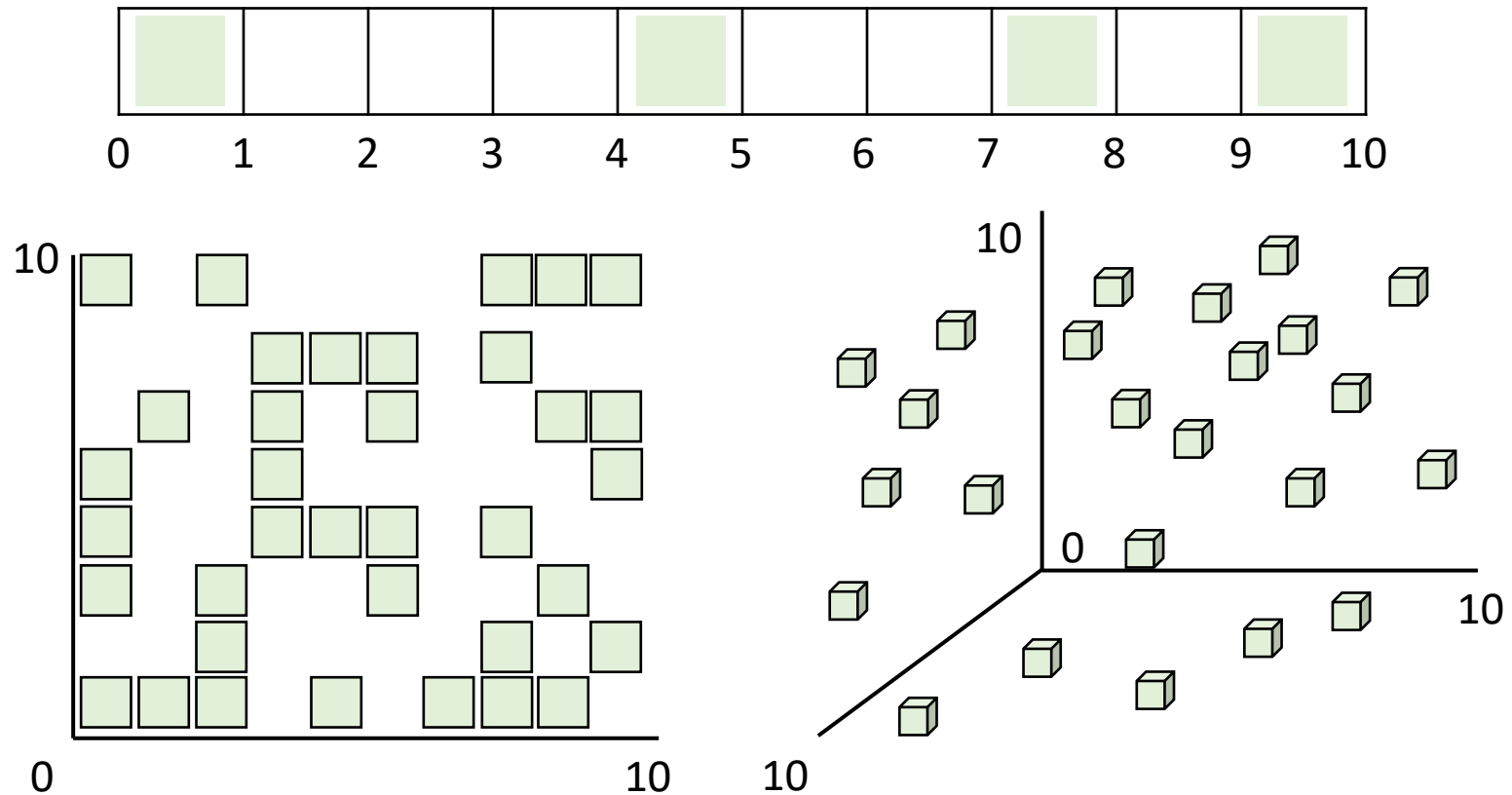


데이터 패턴(군집 등)을 감지해 중요한 정보 획득 가능

차원 축소(Dimensionality Reduction)

■ 차원의 저주(curse of dimensionality)

▶ 차원이 커질수록 해당 차원을 표현하기 위해 필요한 데이터가 급격히 많아짐

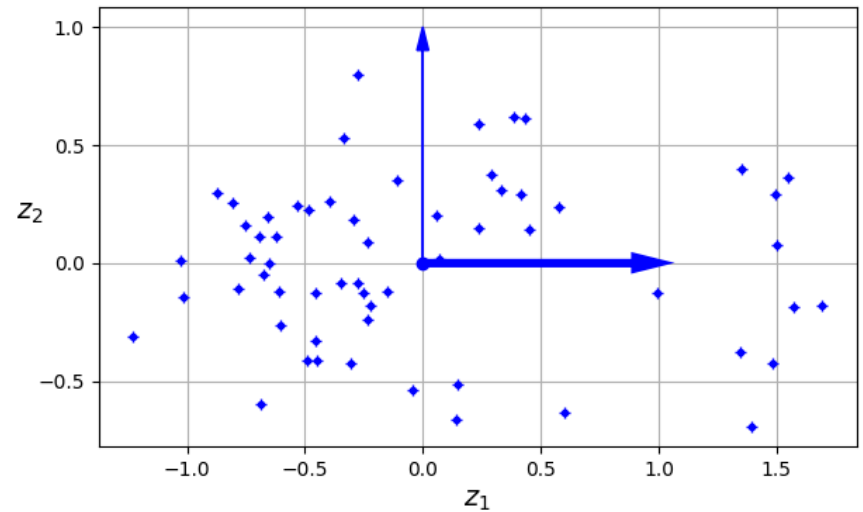
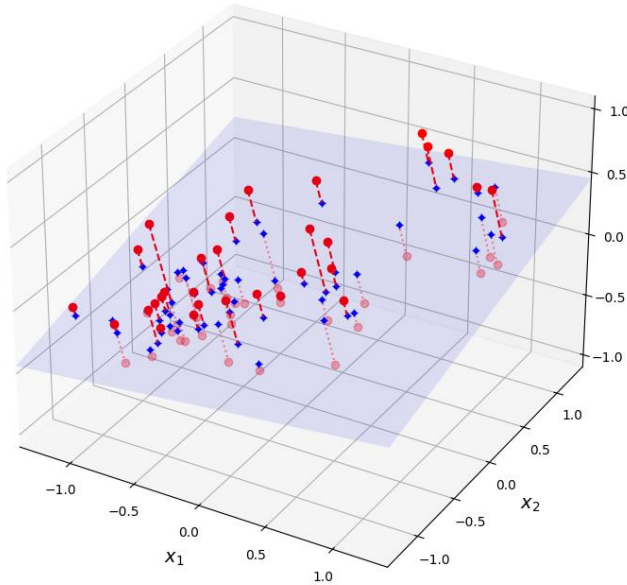


차원이 커질수록 과적합(overfitting) 될 가능성이 높아짐

차원 축소를 위한 접근법(1)

투영(projection)

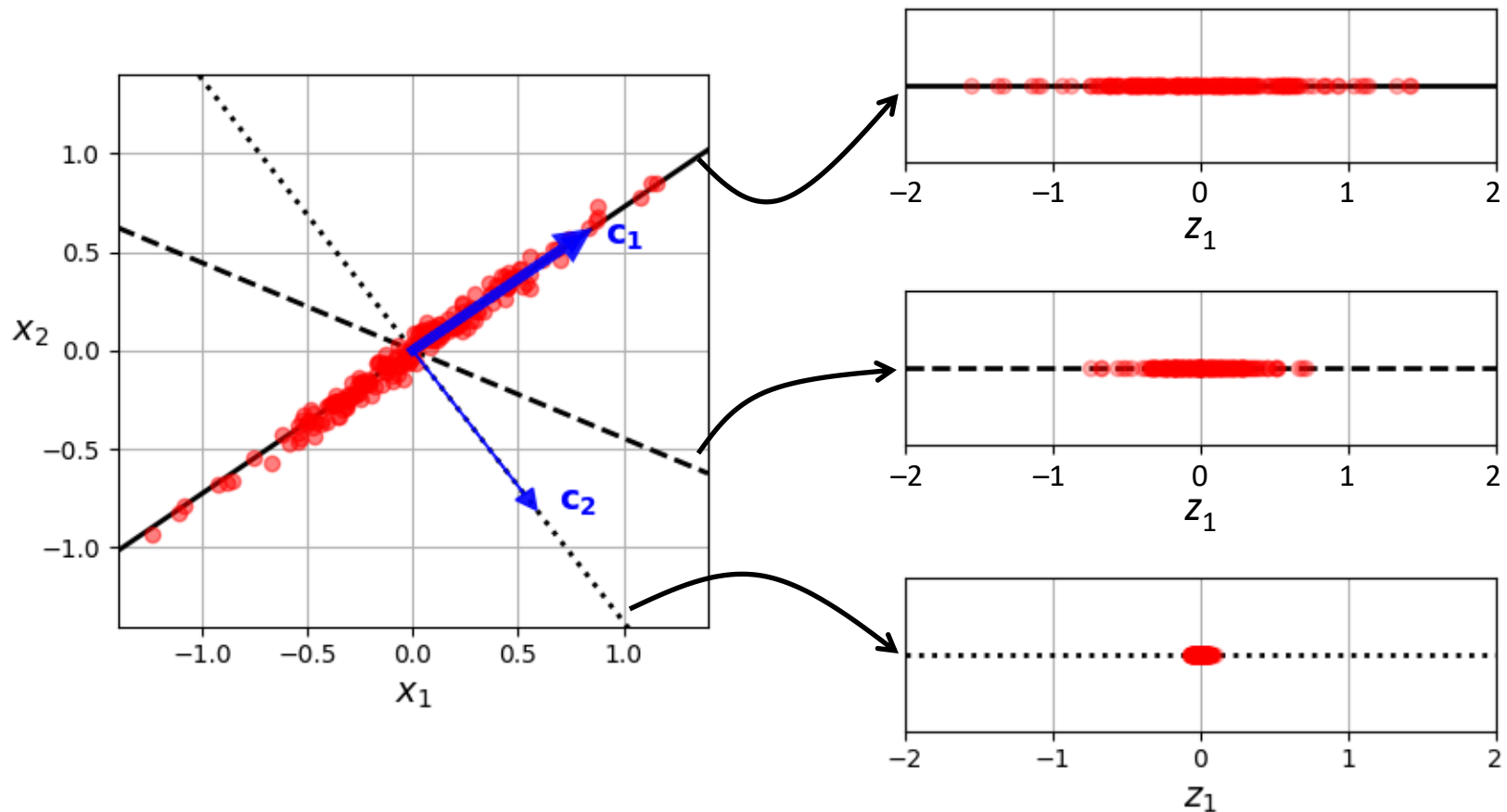
- ▶ 고차원 공간의 데이터 샘플을 고차원 공간 안 저차원 부분 공간(subspace)으로 매핑



차원 축소 알고리즘: PCA

주성분 분석(PCA, principal component analysis)

▶ 데이터 분산이 가장 큰 방향을 찾고, 그 방향의 축으로 데이터를 변환하는 알고리즘

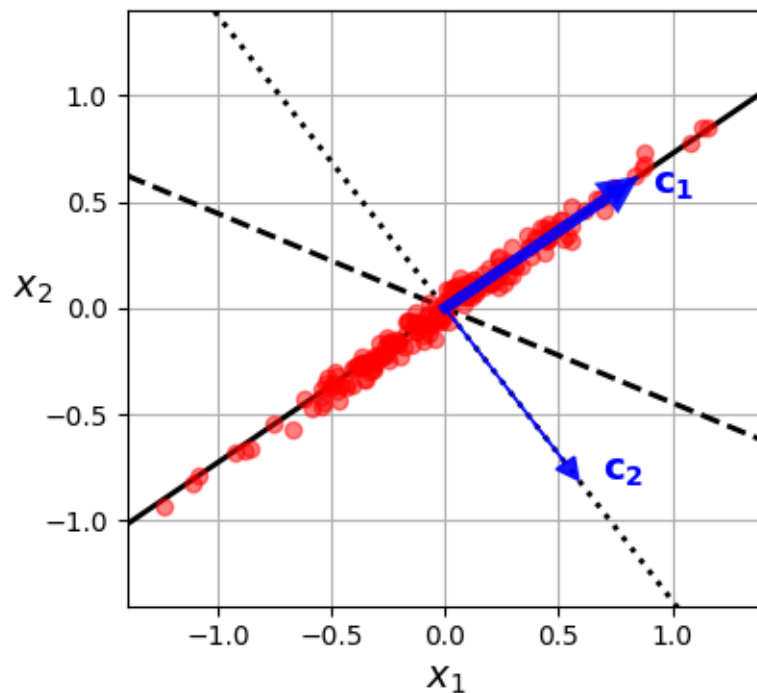


주성분(PC, principal component)

▶ 데이터가 투영되는 축($c_1, c_2, \dots, c_n$)

▶ 주성분 탐색

- 분산이 최대인 축 찾기 $\rightarrow c_1$
- 첫 번째 축에 직교(orthogonal)하고 남은 분산을 최대한 보존하는 두 번째 축 찾기 $\rightarrow c_2$
- 두 번째 축에 직교하고 ... $\rightarrow$ 특징 수(차원)만큼 반복



주성분 단위 벡터 행렬

$$V = \begin{pmatrix} | & | & \dots & | \\ c_1 & c_2 & & c_n \\ | & | & & | \end{pmatrix}$$

차원 축소 알고리즘: PCA

투영(projection)

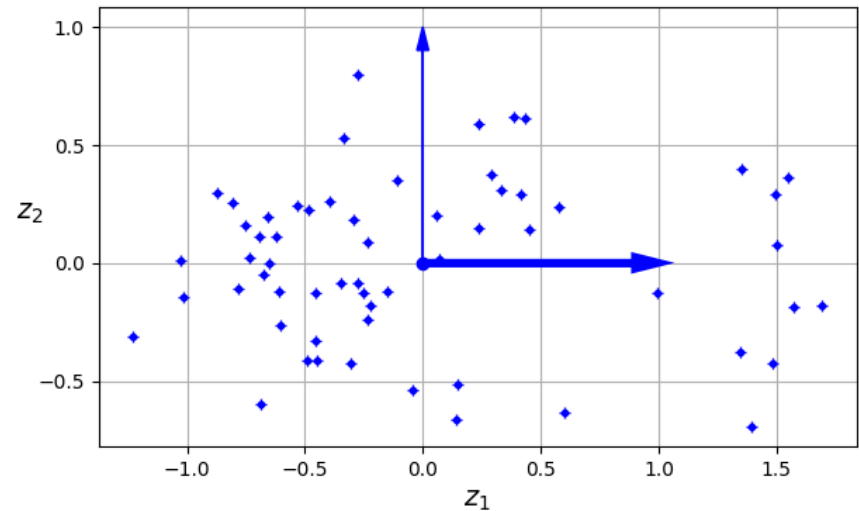
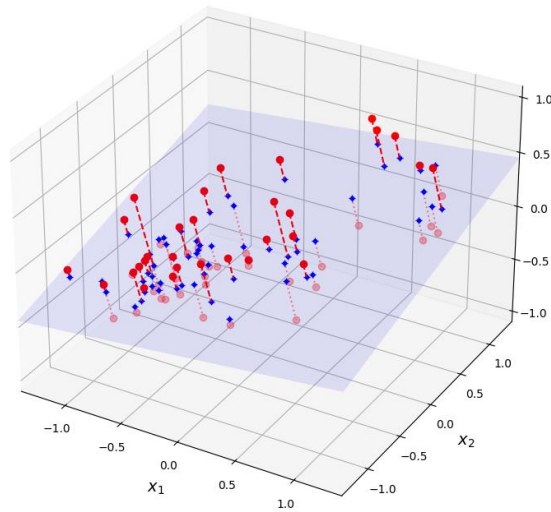
▶ 기존 특징 행렬(X)과 주성분 단위 벡터로 이루어진 행렬(V)의 곱으로 차원 축소

$$X \in \mathbb{R}^{60 \times 3}$$

$$V = \begin{pmatrix} | & | \\ \mathbf{c}_1 & \mathbf{c}_2 \\ | & | \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$



$$Z = XV \in \mathbb{R}^{60 \times 2}$$



차원 축소 알고리즘: PCA

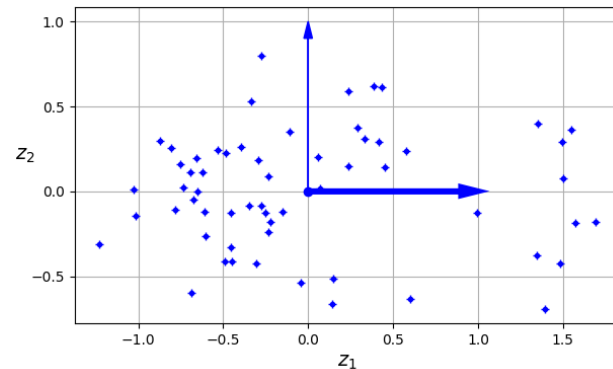
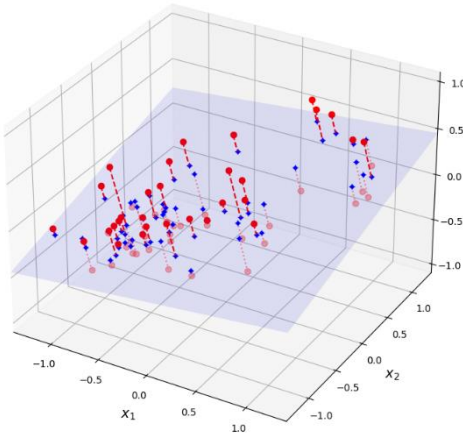
사이킷런 클래스

▶ PCA(n_components = ____)

- n_components : 변환하고자 하는 차원의 특징 수 (정수 입력)

<2차원으로 축소 >

```
from sklearn.decomposition import PCA  
  
pca = PCA(n_components=2)  
X2D = pca.fit_transform(X)
```



사이킷런 클래스

▶ PCA(n_components = ____)

- PCA 객체 fit 이후 각 주성분이 포함하는 분산 비율을 확인할 수 있음
- 실수형 n_components (0.0~1.0): 데이터의 전체 분산을 몇 %까지 보존할지

<주성분이 포함하는 분산 비율 확인>

```
>>> pca.explained_variance_ratio_  
array([0.7578477, 0.15186921])
```

<전체 분산의 95%를 설명할 수 있을 때까지 축소>

```
from sklearn.decomposition import PCA  
  
pca = PCA(n_components=0.95)  
X2D = pca.fit_transform(X)
```

! 최적의 n_components?

▶ 하이퍼파라미터 튜닝을 통해 찾을 수 있음

- GridSearchCV, RandomizedSearchCV를 이용

<하이퍼파라미터 튜닝을 통한 PCA 최적 차원 결정>

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))

param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}

rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                                random_state=42)

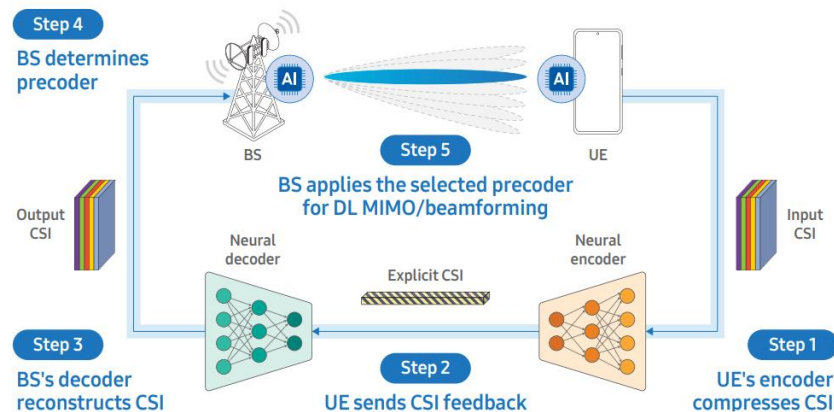
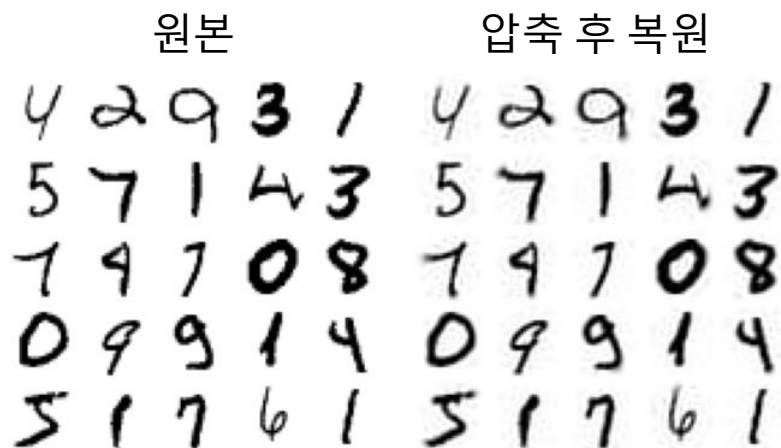
rnd_search.fit(X_train[:1000], y_train[:1000])
```

차원 축소 알고리즘: PCA

데이터 재구성

▶ 차원 축소된 특징을 다시 복구할 수 있음

- fit된 PCA 객체에 `.inverse_transform()` 메서드 사용

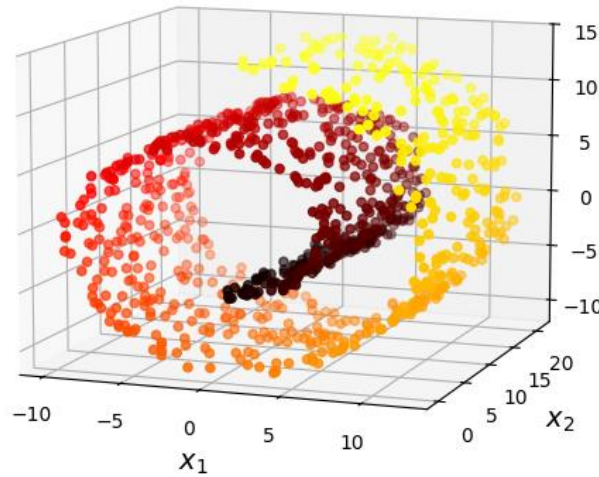
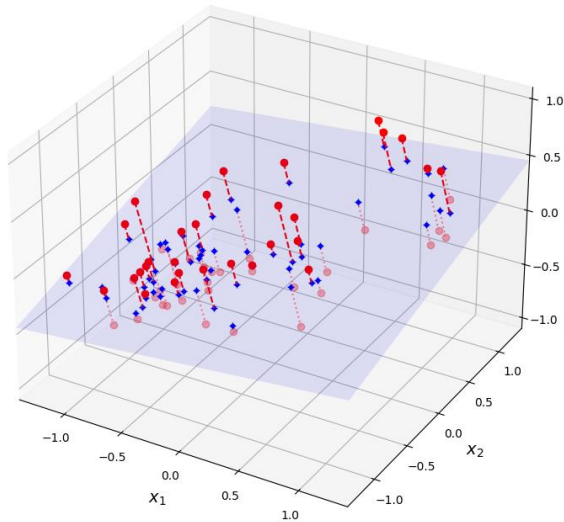


[출처] Samsung Research, "AI-Native & Sustainable Communication", 2025

차원 축소를 위한 접근법(2)

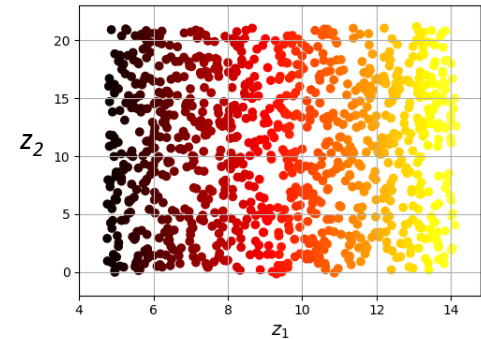
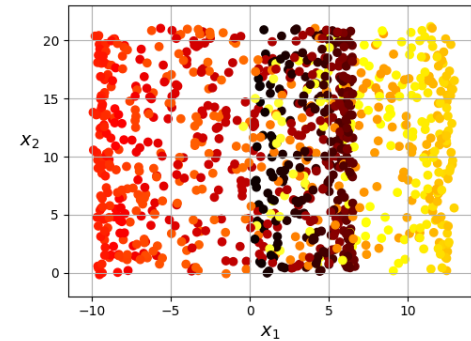
■ 매니폴드(manifold)

▶ 고차원 데이터가 실제로 분포하는 저차원의 구조



투영

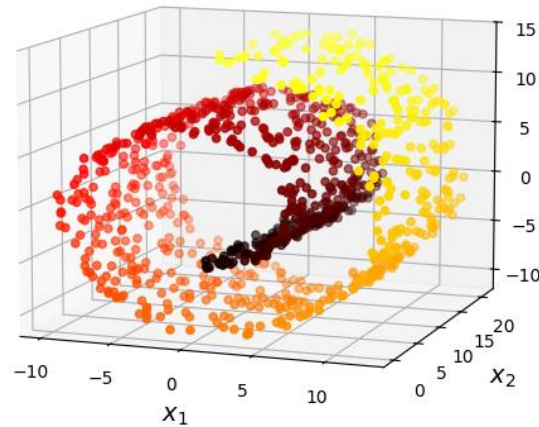
재매핑



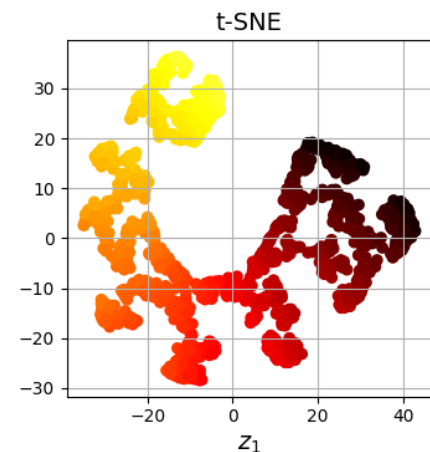
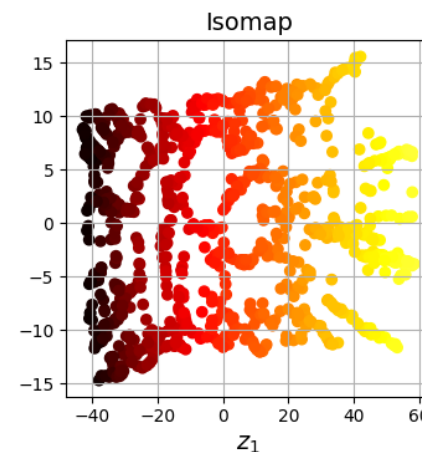
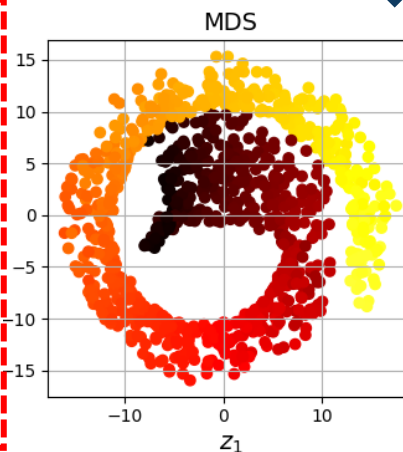
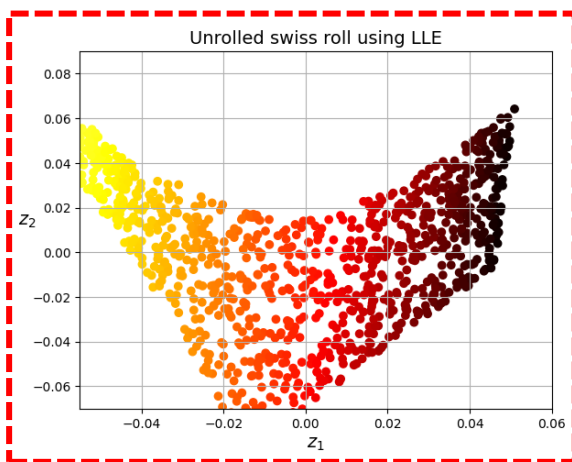
차원 축소를 위한 접근법(2)

■ 매니폴드 학습(manifold learning)

▶ 고차원 데이터의 숨겨진 저차원 구조(매니폴드)를 찾아 표현하는 차원 축소 방법



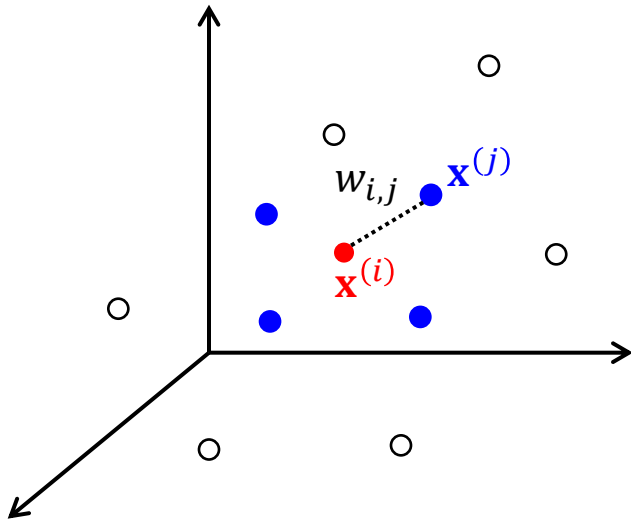
↓ 펼침



차원 축소 알고리즘: LLE

지역 선형 임베딩(LLE, locally linear embedding)

▶ 고차원 공간에서 각 샘플이 주변 이웃들과 어떤 선형 관계를 갖는지 학습



$$\mathbf{x}^{(i)} \approx \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)}$$

$$w_{i,j} = 0,$$

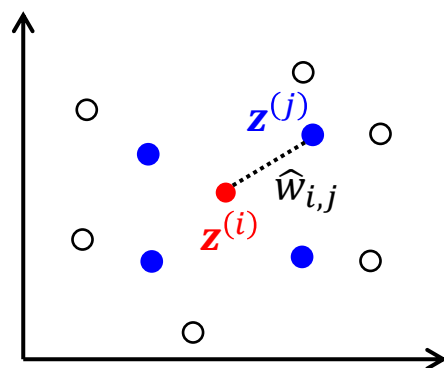
$\mathbf{x}^{(j)}$ 가 $\mathbf{x}^{(i)}$ 의 최근접 이웃 k 개 중 하나가 아닐 때

$$\hat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2$$

$$\text{단, } \sum_{j=1}^m w_{i,j} = 1 \quad (i = 1, 2, \dots, m \text{ 일 때})$$

지역 선형 임베딩(LLE, locally linear embedding)

- ▶ 지역적인 관계가 가장 잘 보존되는 훈련 세트의 저차원 표현을 찾음



$$\mathbf{z}^{(i)} \approx \sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)}$$

$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$

지역적 관계를 보존하는 차원 축소

차원 축소 알고리즘: LLE

사이킷런 클래스

► LocallyLinearEmbedding(n_components = __, n_neighbors = __)

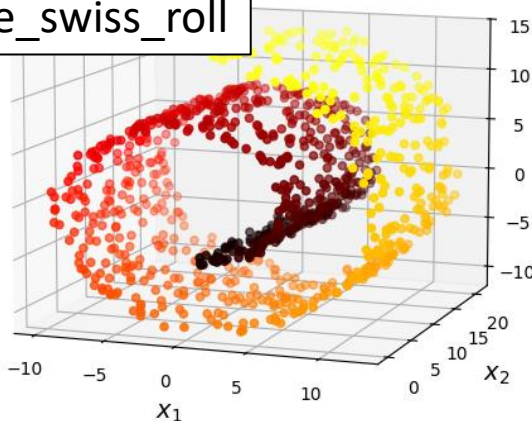
- n_components : 변환하고자 하는 차원의 특징 수 (정수 입력)
- n_neighbors : 가중치 계산 시 고려할 최근접 이웃 개수 (정수 입력)



```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding

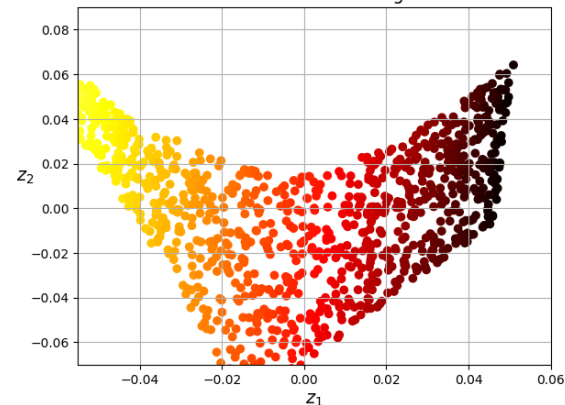
X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)
```

make_swiss_roll



LLE

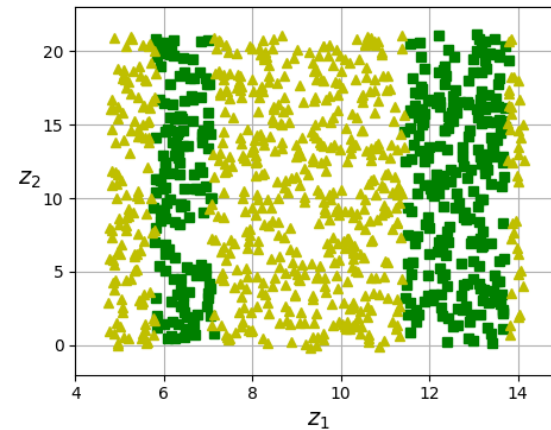
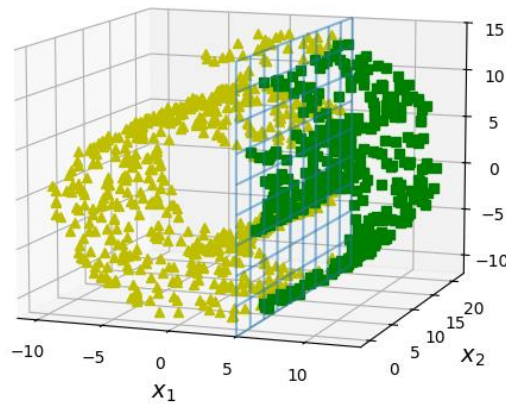
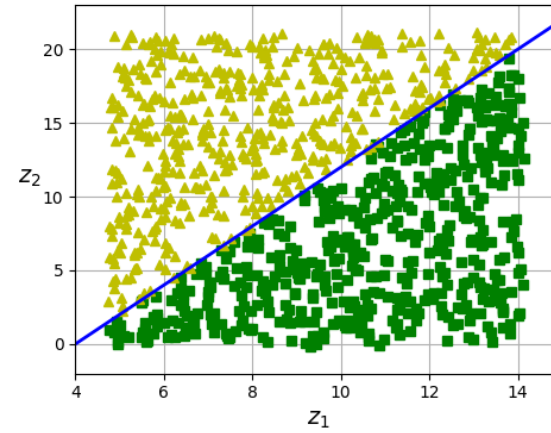
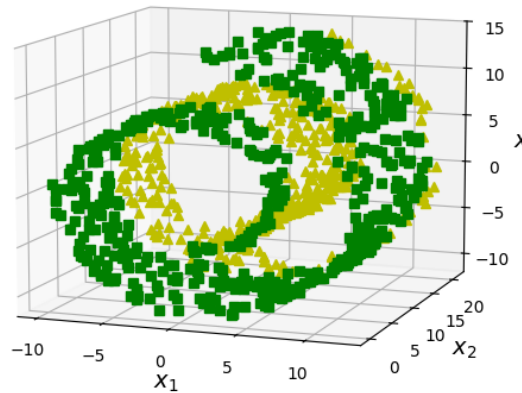
Unrolled swiss roll using LLE



차원 축소의 효과

■ 차원 축소의 효과

- ▶ 훈련 속도 향상, 데이터 시각화에 적합
- ▶ 모델의 성능이 개선되는 것은 아님



▮ 차원 축소의 필요성

- ▶ 차원의 저주

▮ 차원 축소 알고리즘

- ▶ 주성분 분석(PCA)
- ▶ 지역 선형 임베딩(LLE)

▮ 차원 축소의 효과

- ▶ 훈련 속도 향상
- ▶ 데이터 시각화 개선
- ▶ 모델 성능이 향상되는 것은 아님